



Sentencias condicionales

Clase

Codificar una rutina utilizando estructuras condicionales y expresiones booleanas para resolver un problema de baja complejidad de acuerdo al lenguaje python.

- Unidad 1: El lenguaje Python
- Unidad 2: Sentencias básicas del lenguaje Python
- Unidad 3: Sentencias condicionales
- Unidad 4: Funciones y módulos
- Unidad 5: Estructura de datos en Python
- Unidad 6: Secuencias iterativas
- Unidad 7: Conceptos básicos de orientación a objeto en Python



Te encuentras aquí



¿Qué aprenderás en esta sesión?

- *Identifica los tipos de estructura condicional, así como su utilidad en la confección de una rutina de código.*
- *Elabora expresiones booleanas utilizando operadores booleanos y de comparación para resolver un problema de baja complejidad.*
- *Utiliza estructuras de control de flujo de acuerdo a la sintaxis del lenguaje python para resolver el problema planteado.*

Imagina que estás desarrollando un videojuego donde el personaje principal debe decidir si cruzar un puente. ¿Qué condiciones evaluarías para tomar esta decisión? Por ejemplo: ¿el estado del puente, la presencia de enemigos, el nivel de salud del personaje? ¿Cómo expresarías estas condiciones en lenguaje cotidiano?"



`/* Sentencias condicionales */`

Sentencias condicionales

Qué son y para qué sirven

Imagina que estás decidiendo qué ropa usar. Tu decisión depende del clima:

- Si está lloviendo, llevas paraguas.
- Si hace frío, llevas abrigo.
- Si hace calor, llevas ropa ligera.

Imagina que estás usando una aplicación de mensajería. Cuando llega un mensaje nuevo, el programa debe tomar una decisión:

- Si la app está abierta, muestra el mensaje directamente.
- Si la app está cerrada, envía una notificación.

Las sentencias condicionales son como estas "decisiones" en nuestro código. Permiten que nuestro programa tome diferentes caminos según ciertas condiciones.

Sentencias condicionales

Estructura básica

Analicemos el siguiente ejemplo: Salir a jugar basado en si está lloviendo o no.

```
esta_lloviendo = True

if esta_lloviendo:
    print("Mejor me quedo en casa")
```

- La palabra **if** que indica el inicio de la condición.
- La condición que queremos evaluar, en este caso **esta_lloviendo**
- Los dos puntos **:** que indican el inicio del bloque de código.
- La indentación (espacios) que indica qué código pertenece al if.

Sentencias condicionales

Importancia

Las sentencias condicionales son la herramienta que nos permite:

- Controlar el flujo del programa.
- Validar datos de entrada.
- Tomar decisiones basadas en múltiples condiciones.
- Combinar operadores booleanos y de comparación para lógica más compleja.

La clave es entender que:

- La condición siempre debe ser algo que pueda ser verdadero o falso.
- El código indentado solo se ejecuta si la condición es verdadera.
- Si la condición es falsa, Python simplemente sigue con el resto del programa.

Esta es la base sobre la que construiremos estructuras más complejas como if-else y if-elif-else.

Sentencias condicionales

La sentencia if

La sentencia if es como un guardián que controla una puerta: solo deja pasar (ejecutar código) cuando se cumple una condición. Es la forma más básica de tomar decisiones en nuestro programa.

```
# Supongamos que la edad ingresada es 20  
edad = 20  
  
# Verificamos si es mayor de edad  
if edad >= 18:  
    print("Eres mayor de edad")
```

¡Manos a la obra! ¡Aventura
en el mundo de los
videojuegos 🎮!

{desafío}
latam_



¡Aventura en el mundo de los videojuegos!

Ejercicio guiado

Vamos a crear un programa que determine si un jugador puede avanzar al siguiente nivel de un videojuego. ¡Será como crear nuestra propia aventura!

El objetivo es crear un programa que:

1. Reciba los puntos del jugador.
2. Verifique si tiene suficientes puntos para avanzar.
3. Muestre mensajes épicos según su desempeño.



¡Aventura en el mundo de los videojuegos!

Ejercicio guiado

1. Crear la introducción.

```
# Creamos archivo: aventura_gaming.py
print("🎮 AVENTURA GAMING: ¡EL PORTAL MÁGICO! 🎮")
print("=====")
print("¿Tienes lo necesario para cruzar el portal?")
print("Necesitas 100 puntos para avanzar al siguiente nivel")
print()
```

2. Obtener puntos del jugador.

```
# Pedimos los puntos al jugador
puntos = int(input("¡Aventurero! ¿Cuántos puntos has conseguido?: "))
```



¡Aventura en el mundo de los videojuegos!

Ejercicio guiado

3. Verificar el nivel alcanzado.

```
# Verificamos si puede avanzar
if puntos >= 100:
    print("★ ¡PORTAL ABIERTO! ★")
    print("Has desbloqueado el siguiente nivel")
```

4. Agregar logros especiales.

```
# Agregamos Logros según Los puntos
if puntos >= 150:
    print("★ ¡LEGENDARIO! ★")
    print("¡Has conseguido el amuleto dorado!")

if puntos >= 120:
    print("🔥 ¡INCREÍBLE! 🔥")
    print("¡Has conseguido una espada mágica!")

if puntos >= 100 and puntos < 120:
    print("✨ ¡BIEN HECHO! ✨")
    print("¡Has conseguido una poción de energía!")
```



¡Aventura en el mundo de los videojuegos!

Resultado

```
🎮 AVENTURA GAMING: ¡EL PORTAL MÁGICO! 🎮  
=====
```

¿Tienes lo necesario para cruzar el portal?
Necesitas 100 puntos para avanzar al siguiente nivel

★ ¡PORTAL ABIERTO! ★

Has desbloqueado el siguiente nivel

🔥 ¡INCREÍBLE! 🔥

¡Has conseguido una espada mágica!

Resultado

130

Sentencias condicionales

La sentencia if-else

Imagina que estás jugando un videojuego. Al enfrentar un enemigo, solo tienes dos opciones: si tienes una espada, luchas; si no la tienes, huyes. Esto es exactamente para lo que sirve if-else: manejar dos alternativas posibles.

Estructura básica:

```
# Supongamos que la edad ingresada es 20  
edad = 20  
  
# Verificamos si es mayor de edad  
if edad >= 18:  
    print("Eres mayor de edad")  
else:  
    print("Eres menor de edad")
```

¡Manos a la obra! Verificación de edad para un evento



Verificación de edad para un evento

Ejercicio guiado

Escribe un programa que verifique si una persona puede entrar a un evento, considerando que la edad mínima es de 18 años.

1. Crear la introducción del programa:

```
# Introducción
print("🎉 BIENVENIDO AL EVENTO 🎉")
print("Para ingresar al evento debes tener al menos 18 años.")
```



Verificación de edad para un evento

Ejercicio guiado

2. Pedir la edad al usuario:

```
# Pedir la edad al usuario
edad = int(input("Por favor, ingresa tu edad: "))
```

3. Usar la sentencia if-else para verificar la edad:

```
# Verificar la edad usando if-else
if edad >= 18:
    print("✅ ¡Puedes ingresar al evento! Disfruta 🎉")
else:
    print("❌ Lo siento, no puedes ingresar. Debes tener al menos 18 años.")
```



Sentencias condicionales

La sentencia *if-elif-else*

Imagina que estás en una cafetería en Chile y decides qué tipo de café puedes comprar según el dinero que tienes:

- Si tienes más de \$3.000, puedes comprar un café grande.
- Si tienes entre \$1.500 y \$3.000, puedes comprar un café mediano.
- Si tienes menos de \$1.500, solo puedes comprar un café pequeño.

{desafío}
latam_

Este proceso de tomar decisiones en función de una condición es similar a cómo funciona la sentencia condicional **if-elif-else** en Python.

Esta permite elegir diferentes caminos dependiendo de las condiciones que se cumplan.

```
dinero = 2000 # Supongamos que tienes $2.000

if dinero > 3000:
    print("Puedes comprar un café grande")
elif 1500 <= dinero <= 3000:
    print("Puedes comprar un café mediano")
else:
    print("Solo puedes comprar un café pequeño")
```

¡Manos a la obra! Control de temperatura

{desafío}
latam_



Control de temperatura

Ejercicio

Crea un programa que reciba una temperatura y muestre diferentes mensajes:

- Si es mayor a 37.5: "Tienes fiebre"
- Si está entre 37 y 37.5: "Temperatura elevada"
- Si está entre 36 y 37: "Temperatura normal"
- Si es menor a 36: "Temperatura baja"

Instrucciones paso a paso:

1. Crea un archivo `control_temperatura.py`
2. Solicita la temperatura al usuario.
3. Implementa las condiciones usando if-elif-else.
4. Muestra los mensajes apropiados.



`/* Operadores booleanos */`

Operadores booleanos

¿Qué son?

Imagina que eres juez/a en un torneo de deportes. Para decidir si un equipo puede participar, necesitas verificar varias condiciones:

- que todos los jugadores tengan la edad correcta **Y** que hayan entregado sus certificados médicos, **O** que tengan un permiso especial.

En Python, los operadores booleanos funcionan como ese juez; nos ayudan a tomar decisiones basadas en múltiples condiciones.

Operadores booleanos

Operador and

El operador **and** es como un "juez exigente": todas las condiciones deben ser verdaderas para que el resultado sea verdadero.

Por ejemplo, para conducir un auto necesitas:

- Tener licencia **Y**
- Ser mayor de edad **Y**
- No haber consumido alcohol.

```
edad = 19
tiene_licencia = True
esta_sobrio = True

puede_conducir = edad >= 18 and tiene_licencia and esta_sobrio
print(f"¿Puede conducir? {puede_conducir}")
```

La variable **puede_conducir** evalúa si todas las condiciones son verdaderas, y si lo son, devuelve **True**, lo que significa que la persona puede conducir. En este caso, como **la edad es 19, la persona tiene licencia (True), y está sobria (True), el resultado será True.**

Operadores booleanos

Operador or

El operador **or** es como un "juez flexible": si al menos una condición es verdadera, el resultado es verdadero.

Por ejemplo, para entrar a un parque de diversiones necesitas:

- Medir más de 1.20m de altura **O**
- Ir acompañado de un adulto.

```
altura = 1.15
con_adulto = True

puede_entrar = altura >= 1.20 or con_adulto
print(f"¿Puede entrar al juego? {puede_entrar}")
```

En este caso, aunque la persona no cumple con la altura mínima ($1.15 < 1.20$), como está acompañada por un adulto (`con_adulto = True`), la condición global se evalúa como verdadera.

Operadores booleanos

Operador not

El operador **not** es como un "interruptor": cambia verdadero por falso y viceversa.

Por ejemplo, si NO está lloviendo, podemos salir a jugar.

```
esta_lloviendo = False
podemos_salir = not esta_lloviendo
print(f"¿Podemos salir a jugar? {podemos_salir}")
```

En este caso se evalúa si se puede salir a jugar dependiendo de si está lloviendo.

La variable **esta_lloviendo es False**, lo que significa que no está lloviendo.

Usando el operador lógico **not**, se invierte el valor de esta variable, por lo que **podemos_salir se vuelve True**, indicando que sí se puede salir.

El resultado se imprime con la respuesta "¿Podemos salir a jugar? True".

¡Manos a la obra! Sistema de acceso a biblioteca



Sistema de acceso a biblioteca

Ejercicio guiado

Crearemos un sistema que determine si un estudiante puede acceder a la biblioteca basado en múltiples condiciones.

1. Crear el archivo biblioteca_acceso.py
2. Definir las variables necesarias.

```
# Pedimos la información necesaria
carnet = input("¿Tiene carnet de biblioteca? (s/n): ")
tiene_carnet = carnet.lower() == 's'

multas = input("¿Tiene multas pendientes? (s/n): ")
tiene_multas = multas.lower() == 's'

horario = int(input("¿Qué hora es? (formato 24hrs, ej: 14): "))
```



Sistema de acceso a biblioteca

Ejercicio guiado

3. Crear las condiciones de acceso.

```
# Verificamos el horario (biblioteca abierta de 8 a 20)  
horario_permitido = horario >= 8 and horario < 20  
  
# Verificamos el estado del estudiante  
estado_correcto = tiene_carnet and not tiene_multas  
  
# Combinamos todas las condiciones  
acceso_permitido = horario_permitido and estado_correcto
```



Sistema de acceso a biblioteca

Ejercicio guiado

4. Mostrar resultado.

```
# Mostrar el resultado
print("\nResultado del análisis de acceso:")
print("-" * 30)

if acceso_permitido:
    print("Acceso PERMITIDO")
else:
    print("Acceso DENEGADO")
    print("\nRazones:")
    if not horario_permitido:
        print("- Fuera de horario de atención (8:00 a 20:00)")
    if not tiene_carnet:
        print("- No tiene carnet de biblioteca")
    if tiene_multas:
        print("- Tiene multas pendientes")
```



/* Operadores de comparación */

Operadores de comparación

¿Qué son?

Son símbolos especiales que nos permiten comparar dos valores y obtener un resultado booleano (True o False).

Operadores de comparación

Mayor que (>) y Mayor o igual que (>=)

- El operador "**mayor que**" (>) se usa cuando queremos verificar si un número es estrictamente más grande que otro.
- El operador "**mayor o igual que**" (>=) se usa cuando queremos verificar si un número es más grande o exactamente igual a otro.
 - Por ejemplo, cuando comprobamos la edad para entrar a una discoteca, necesitamos ser mayores de 18 años, o cuando queremos saber si un estudiante aprobó con una nota mayor o igual a 4.0.

```
# 1. Mayor que (>) y Mayor o igual que (>=)
```

```
edad = 18
```

```
print("\nEjemplos con edad = 18:")
```

```
print(f"¿Es mayor que 16? {edad > 16}") # True
```

```
print(f"¿Es mayor que 18? {edad > 18}") # False
```

```
print(f"¿Es mayor o igual que 18? {edad >= 18}") # True
```

```
print(f"¿Es mayor o igual que 20? {edad >= 20}") # False
```

Operadores de comparación

Menor que (<) y Menor o igual que (<=)

- El operador "**menor que**" (<) se usa cuando queremos verificar si un número es estrictamente más pequeño que otro.
- El operador "**menor o igual que**" (<=) se usa cuando queremos verificar si un número es más pequeño o exactamente igual a otro.
- Por ejemplo, cuando verificamos si un producto está en oferta (precio menor a \$1000), o cuando queremos saber si un niño puede subir a un juego (altura menor o igual a 1.2 metros).

```
# 2. Menor que (<) y Menor o igual que (<=)
precio = 990
print("\nEjemplos con precio = 990:")
print(f"¿Es menor que 1000? {precio < 1000}")      # True
print(f"¿Es menor que 900? {precio < 900}")        # False
print(f"¿Es menor o igual que 990? {precio <= 990}") # True
print(f"¿Es menor o igual que 900? {precio <= 900}") # False
```

Operadores de comparación

Igual que (==)

- El operador "igual que" (==) se usa cuando queremos verificar si dos valores son exactamente iguales.
 - Por ejemplo, cuando verificamos si una contraseña ingresada coincide con la almacenada, o si un código de descuento es válido.

```
# 3. Igual que (==)
contraseña = "abc123"
print("\nEjemplos con contraseña = 'abc123':")
print(f"¿Es igual a 'abc123'? {contraseña == 'abc123'}") # True
print(f"¿Es igual a '123abc'? {contraseña == '123abc'}") # False
```

Operadores de comparación

Distinto que (!=)

- El operador "distinto que" (!=) se usa cuando queremos verificar si dos valores son diferentes.
 - Por ejemplo, cuando queremos asegurarnos que un usuario eligió una contraseña nueva diferente a la anterior.

```
# 4. Distinto que (!=)
color = "rojo"
print("\nEjemplos con color = 'rojo':")
print(f"¿Es distinto de 'verde'? {color != 'verde'}") # True
print(f"¿Es distinto de 'rojo'? {color != 'rojo'}")  # False
```

/* Paréntesis y expresiones booleanas */

Paréntesis y expresiones booleanas

Los paréntesis en expresiones booleanas son fundamentales porque:

- Ayudan a organizar y agrupar condiciones.
- Modifican el orden de evaluación.
- Hacen el código más legible.

Veamos un ejemplo sin paréntesis:

```
if edad >= 18 and estado == "soltero" or estado == "divorciado":  
    print("Califica")
```

Y uno con paréntesis:

```
if edad >= 18 and (estado == "soltero" or estado == "divorciado"):  
    print("Califica")
```

Los paréntesis nos ayudan a especificar exactamente qué queremos evaluar primero.

¡Manos a la obra! Sistema de clasificación de Torneos de Videojuegos 🎮



Sistema de Clasificación de Torneos de Videojuegos

Ejercicio

1. Configuración Inicial
 - Crea un nuevo archivo llamado torneo_videojuegos.py
 - Agrega un mensaje de bienvenida usando print()
2. Recolección de Datos
 - Solicita la siguiente información al usuario:
 - Edad (número entero)
 - Puntaje actual (0-100)
 - Si tiene equipo (si/no)
 - Horas de práctica semanales
 - Número de torneos previos
 - Usa input() y convierte los datos al tipo apropiado



Sistema de Clasificación de Torneos de Videojuegos

Ejercicio

3. Verificación de Elegibilidad

- Crea una variable `es_elegible` que combine:
 - Edad mínima de 13 años
 - Puntaje mínimo de 20 puntos
- Usa el operador `and` y paréntesis para agrupar las condiciones

4. Estructura de Categorización

- Implementa la primera condición usando `if not es_elegible`
- Crea un bloque `else` para procesar jugadores elegibles
- Dentro del `else`, implementa la lógica de categorización usando `if-elif-else`:
 - Profesional: (puntaje ≥ 80 y horas ≥ 10) o (torneos ≥ 5 y puntaje ≥ 70)
 - Semi-profesional: puntaje ≥ 60 y (horas ≥ 5 o torneos ≥ 2)
 - Amateur Avanzado: puntaje ≥ 40
 - Amateur: resto de casos



Sistema de Clasificación de Torneos de Videojuegos

Ejercicio

5. Determinación del Modo de Participación
 - Dentro del bloque `else` principal
 - Implementa la lógica para determinar si el jugador competirá en equipo o individual
 - Usa los operadores `and`, `or` y `not` según corresponda
6. Mostrar Resultados
 - Imprime los resultados usando `print()`
 - Muestra:
 - Categoría asignada
 - Modo de participación
 - Beneficios según la categoría



Sistema de Clasificación de Torneos de Videojuegos

Ejercicio

Sugerencias:

- Escribe primero la estructura básica del programa.
- Prueba cada sección antes de continuar.
- Verifica diferentes combinaciones de datos de entrada.
- Asegúrate de usar paréntesis para hacer las expresiones booleanas claras y precisas.



Sistema de Clasificación de Torneos de Videojuegos

Ejercicio

Ejemplo de ejecución:

```
=== Sistema de Clasificación para Torneo de Videojuegos ===  
Ingrese los datos del jugador para verificar su categoría  
Ingrese la edad del jugador: 15  
Ingrese el puntaje actual (0-100): 85  
¿Tiene equipo? (si/no): si  
Ingrese horas de práctica semanales: 12  
Número de torneos previos: 3  
  
Resultados de la clasificación:  
Categoría asignada: Profesional  
Modo de participación: Competencia en equipo  
Beneficios: Acceso a todas las competencias y sala VIP
```



Resumen

Subtítulo

- Las sentencias condicionales son estructuras fundamentales en Python que permiten que nuestros programas tomen decisiones basadas en condiciones específicas.
- Python nos ofrece tres tipos principales de estructuras condicionales: la sentencia `if` simple, que ejecuta un bloque de código solo cuando se cumple una condición; la estructura `if-else`, que nos permite manejar dos alternativas posibles; y la estructura `if-elif-else`, que facilita la toma de decisiones entre múltiples opciones.
- Para construir estas condiciones, utilizamos operadores de comparación (`<`, `>`, `<=`, `>=`, `==`, `!=`) que nos permiten establecer relaciones entre valores, y operadores booleanos (`and`, `or`, `not`) que nos ayudan a combinar múltiples condiciones.
- Además, el uso adecuado de paréntesis en expresiones booleanas nos permite organizar y priorizar la evaluación de condiciones complejas, mejorando tanto la claridad del código como su correcta ejecución.

En la sesión de hoy, hemos aprendido sobre diferentes estructuras condicionales y cómo combinarlas. Pensando en tu día a día, ¿podrías identificar tres situaciones donde tomas decisiones que podrían representarse usando las estructuras if, if-else o if-elif-else? ¿Cómo las expresarías en Python?"





Próxima sesión...

Codificar una rutina utilizando funciones preconstruidas, funciones personalizadas o de un módulo para resolver un problema de baja complejidad de acuerdo al lenguaje python.

{desafío}
latam_

*Academia de
talentos digitales*

